

Fortgeschrittene Lineare Algebra und Analytische Geometrie

03 $A = LU$ und weitere Zerlegungen

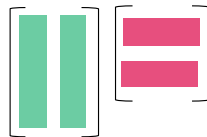
Martin Zaefferer

Quellen

- Strang. 2020. Linear Algebra for Everyone. Wellesley-Cambridge Press.
 - Scan von Übungsaufgaben liegt im Moodle
- Deisenroth, Faisal, Ong. 2021. Mathematics for Machine Learning. Cambridge University Press
 - Enthält auch Übungsaufgaben
- Morten Hjorth-Jensen. 2022. Computational Physic.
 - <https://github.com/CompPhysics/ComputationalPhysics>
- Kenji Hiranabe. 2023. The Art of Linear Algebra.
 - <https://github.com/kenjihiranabe/The-Art-of-Linear-Algebra>

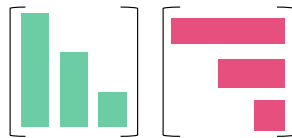
Zerlegungen

$$A = CR$$



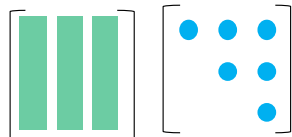
Linear unabhängige Spalten in C
Red. Zeilenstufenform in R
Spaltenrang = Zeilenrang

$$A = LU$$



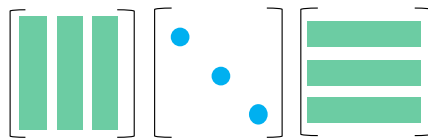
LU Zerlegung mit Gaußverfahren
(Untere Dreiecksm.) (Obere Dreiecksm.)

$$A = QR$$



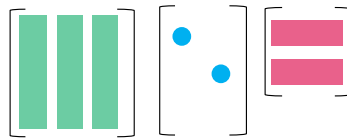
QR Zerlegung mit
Gram-Schmidt Orthogonalisierung
Orthogonalmatrix Q und Dreiecksmatrix R

$$S = Q\Lambda Q^T$$



Eigenwertzerlegung
einer symmetrischen Matrix S
Eigenvektoren in Q , Eigenwerte in Λ

$$A = U\Sigma V^T$$



Singulärwertzerlegung aller Matrizen A
Singularwerte in Σ
Singularvektoren in U, V

$$A = LU$$

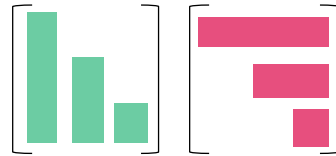


Bild: Kenji Hiranabe. 2023. The Art of Linear Algebra.

LU Zerlegung: $A = LU$

LU -Zerlegung (manchmal auch LR-Zerlegung):

- L : untere Dreiecksmatrix (L für lower)
 - Mit Einsen auf Hauptdiagonale
- U : obere Dreiecksmatrix (U für upper)
- Anmerkung: L und U sind nicht unbedingt eindeutig
 - Eine Matrix A kann also (möglicherweise) in verschiedene Paare L, U zerlegt werden

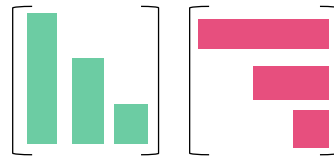


Bild: Kenji Hiranabe. 2023. The Art of Linear Algebra.

Beispiel

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix}$$

$A = LU$... warum?

LU Zerlegung kann als Grundlage anderer Algorithmen dienen, z.B. zum Lösen von LGS:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = w_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = w_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = w_3$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = w_4.$$

- Dieses LGS kann mit LU (und anderen Schritten) gelöst werden.
- Berechnung der Determinante, Inverse, ...

Im Grunde: 'Strukturierte' Umsetzung des Gauß-Verfahrens

$A = LU$... wann?

- Nur für quadratische Matrizen $A \in \mathbb{R}^{n \times n}$
- Matrix A kann in LU zerlegt werden, wenn $|A_k| \neq 0 \quad \forall k \in \{1, \dots, n\}$, d.h., nicht singulär
 - Hier: A_k ist **führende Untermatrix**, also eine Matrix der ersten k Spalten und Zeilen
- Wenn LU Zerlegung existiert, ist die Determinante ...?

$$|A| = |LU| = |L| |U| = \dots$$

$A = LU$... wann?

- Nur für quadratische Matrizen $A \in \mathbb{R}^{n \times n}$
- Matrix A kann in LU zerlegt werden, wenn $|A_k| \neq 0 \quad \forall k \in \{1, \dots, n\}$, d.h., nicht singulär
 - Hier: A_k ist **führende Untermatrix**, also eine Matrix der ersten k Spalten und Zeilen
 - Anmerkung: Gilt nur für **einfache** Verfahren für LU Zerlegung. Verfahren mit **Pivotisierung** können trotzdem LU erzeugen.
 - Zudem: manchmal einfachere Fälle, keine Pivotisierung notwendig
- Wenn LU Zerlegung existiert, ist die Determinante ...?

$$|A| = |LU| = |L| |U| = u_{11}u_{22} \dots u_{nn}$$

- Wichtiger Nebeneffekt: Determinante ist Produkt der Hauptdiagonalen von U !

$A = LU$... warum?

Vorteile der LU Zerlegung, im Vergleich zu anderen Verfahren

- Effiziente Berechnung zur Lösung von $A\vec{x} = \vec{b}$
 - Relativ schnell, möglicherweise auch mit weniger Speicherbedarf
 - Aber: andere Verfahren könnten je nach Fall auch besser sein
- Gute numerische Stabilität

$A = LU$ für LGS

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4 = w_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4 = w_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4 = w_3$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 + a_{44}x_4 = w_4.$$

LGS, in Matrixschreibweise:

$$A\vec{x} = \vec{w}$$

ergibt mit $A = LU$:

$$A\vec{x} = LU\vec{x} = \vec{w}$$

$A = LU$ für LGS

$$LU\vec{x} = \vec{w}$$

kann schrittweise berechnet werden mit:

$$\vec{y} = L^{-1}\vec{w} \quad \vec{x} = U^{-1}\vec{y}$$

Weil:

$$LU\vec{x} = \vec{w} \tag{1}$$

$$U\vec{x} = L^{-1}\vec{w} = \vec{y} \tag{2}$$

$$L^{-1}\vec{w} = \vec{y} \tag{3}$$

$$\vec{x} = U^{-1} \vec{y} \tag{4}$$



- Aber moment, dann müssen wir *zusätzlich* noch zwei Inverse berechnen!?



- Nein, müssen wir nicht
- Stufenform liegt vor, direkt durch (zweimaliges) einsetzen lösbar

$$L\vec{y} = \vec{w} \quad U\vec{x} = \vec{y}$$

$A = LU$ Beispiel 4D

LGS lösen mittels Vorwärts-Einsetzen

$$y_1 = w_1$$

$$l_{21}y_1 + y_2 = w_2$$

$$l_{31}y_1 + l_{32}y_2 + y_3 = w_3$$

$$l_{41}y_1 + l_{42}y_2 + l_{43}y_3 + y_4 = w_4.$$

und Rückwärtseinsetzen

$$u_{11}x_1 + u_{12}x_2 + u_{13}x_3 + u_{14}x_4 = y_1$$

$$u_{22}x_2 + u_{23}x_3 + u_{24}x_4 = y_2$$

$$u_{33}x_3 + u_{34}x_4 = y_3$$

$$u_{44}x_4 = y_4$$

Beispielrechnung

Mit $A\vec{x} = \vec{b}$,

$$A = LU = \begin{pmatrix} -1 & 0 & -1 & 1 \\ -1 & 2 & 2 & 2 \\ -1 & 0 & 1 & -1 \\ -1 & 0 & 2 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1.5 & 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\text{und } \vec{b} = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 0 \end{pmatrix}$$

Lösung für $\vec{y} = ?$, $\vec{x} = ?$

Beispielrechnung - Ergebnisse

$$\vec{y} = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 0 \end{pmatrix}$$

und

$$\vec{x} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

Beispielrechnung 2

$$A = LU = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 3 & 7 \\ 1 & 3 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 3 & 5 \\ 0 & -3 & -3 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\text{und } \vec{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

Lösung für $\vec{y} = ?$, $\vec{x} = ?$

Ergebnis

$$\vec{y} = (1, 0, 2), \text{ und } \vec{x} = \dots \text{ ⚡}$$

Achtung: Wir wissen bereits, dass A keinen vollen Rang hat. Zudem enthält $\vec{y}$ keine Nullen an der entscheidenden dritten Stelle. Somit keine Lösung möglich!

Beispielrechnung 3

$$A = LU = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 3 & 7 \\ 1 & 3 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 3 & 5 \\ 0 & -3 & -3 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\text{und } \vec{b} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

Lösung für $\vec{y} = ?$, $\vec{x} = ?$

Ergebnis

$$\vec{y} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$U\vec{x} = \vec{y}$$

Mit RZSF: Partikulärlösung

$$\left(\begin{array}{ccc|c} 1 & 3 & 5 & 1 \\ 0 & -3 & -3 & 0 \\ 0 & 0 & 0 & 0 \end{array} \right)$$

$$\left(\begin{array}{ccc|c} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{array} \right)$$

$$\vec{x} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

Allgemeine Lösung, z.B. mit -1 Trick:

$$\begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & -1 \end{pmatrix}$$

$$\{\vec{x} | \vec{x} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + a \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} \text{ mit } a \in \mathbb{R}\}$$

$A = LU$ und die inverse Matrix

Wenn Inverse existiert:

$$AA^{-1} = E$$

Mit LU :

$$LU A^{-1} = E$$

$A = LU$ und die inverse Matrix

Erste Spalte der Inversen:

$$A_1^{-1} = \begin{pmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ \dots \\ a_{n1}^{-1} \end{pmatrix}$$

ergibt LGS

$$LU \begin{pmatrix} a_{11}^{-1} \\ a_{21}^{-1} \\ \dots \\ a_{n1}^{-1} \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ \dots \\ 0 \end{pmatrix}$$

Lösung über Vorwärts-Rückwärts-Einsetzen

$A = LU$ und die inverse Matrix

Entsprechend für zweite Spalte der Inversen:

$$LU \begin{pmatrix} a_{12}^{-1} \\ a_{22}^{-1} \\ \dots \\ a_{n2}^{-1} \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ \dots \\ 0 \end{pmatrix}$$

Und so weiter, für jede Spalte der Inversen.

Pivotisierung (partielle, volle)

- Varianten der LU Zerlegung setzen 'Pivotisierung' ein
- Ziel: stabilere Rechnung durch
 - Tauschung von Zeilen (partiell)
 - Tauschung von Zeilen und Spalten (voll)
- Die Zerlegung ist dann:
 - partielle: $PA = LU$ (oder $A = PLU$)
 - volle: $PAQ = LU$
- Mit Permutationsmatrix P (& Q)

Kein Grundsätzlicher Unterschied in der Anwendung, aber P (& Q) sind wichtig für Interpretation / Verwendung der Matrizen L, U

Bestimmung von L, U ?

- Bestimmung ist u.A. über ZSF mittels Gauß möglich
 - Für uns manuell durchführbar
- In der Praxis:
 - Manuelle Bestimmung über ZSF mit Gauß eher nicht hilfreich
 - Bestimmung mittels Software
 - Schauen wir uns nicht im Detail an
- Wichtig: Verständnis der Nutzung / Anwendung und Vorteile!

Bestimmung mit Software

- Hochspezialisierte Software für effiziente Lösung, siehe z.B.:
 - BLAS (Basic Linear Algebra Subprograms) <https://www.netlib.org/blas/>
 - LAPACK (Linear Algebra PACKage) <https://www.netlib.org/lapack/>
 - Programmiersprache: Fortran
- Wird üblicherweise als Grundlage benutzt in “high-level” Schnittstellen von fast allen Programmiersprachen (R, Python, matlab, ...)
- Vorsicht:
 - Ab und an gibt es *Eigenimplementationen* von Paket-Autoren
 - Manchmal gerechtfertigt/sinnvoll, aber sehr häufig *extrem langsam* im Vergleich zu LAPACK/BLAS

Bestimmung mit Gauß 1

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.I} \\ \text{mit Z.I} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ -1 & 2 & 2 & 2 \\ -1 & 0 & 1 & -1 \\ -1 & 0 & 2 & -1 \end{pmatrix} \begin{array}{l} -1 \cdot I \\ -1 \cdot I \\ -1 \cdot I \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.II} \\ \text{mit Z.II} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} \begin{array}{l} -0 \cdot II \\ -0 \cdot II \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.III} \\ \text{mit Z.III} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} -3/2 \cdot III$$

$$\text{ZSF} = \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Also ...?

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ ? & 1 & 0 & 0 \\ ? & ? & 1 & 0 \\ ? & ? & ? & 1 \end{pmatrix}$$

$$U = \begin{pmatrix} ? & ? & ? & ? \\ 0 & ? & ? & ? \\ 0 & 0 & ? & ? \\ 0 & 0 & 0 & ? \end{pmatrix}$$

Bestimmung mit Gauß 2

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.I} \\ \text{mit Z.I} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ -1 & 2 & 2 & 2 \\ -1 & 0 & 1 & -1 \\ -1 & 0 & 2 & -1 \end{pmatrix} \begin{array}{l} -1 \cdot I \\ -1 \cdot I \\ -1 \cdot I \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.II} \\ \text{mit Z.II} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} \begin{array}{l} \\ -0 \cdot II \\ -0 \cdot II \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.III} \\ \text{mit Z.III} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} -3/2 \cdot III$$

$$\text{ZSF} = \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Wir wissen (einige Folien vorher):

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 1.5 & 1 \end{pmatrix}$$

$$U = \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Bestimmung mit Gauß 3

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.I} \\ \text{mit Z.I} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ -1 & 2 & 2 & 2 \\ -1 & 0 & 1 & -1 \\ -1 & 0 & 2 & -1 \end{pmatrix} \begin{array}{l} -\textcolor{red}{1} \cdot I \\ -\textcolor{red}{1} \cdot I \\ -\textcolor{red}{1} \cdot I \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.II} \\ \text{mit Z.II} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} \begin{array}{l} \\ -\textcolor{green}{0} \cdot II \\ -\textcolor{green}{0} \cdot II \end{array}$$

$$\begin{array}{l} \text{Erzeuge 0} \\ \text{in S.III} \\ \text{mit Z.III} \end{array} \begin{pmatrix} -1 & 0 & -1 & 1 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 3 & -2 \end{pmatrix} -\textcolor{violet}{3/2} \cdot III$$

$$\text{ZSF} = \begin{pmatrix} \textcolor{blue}{-1} & \textcolor{blue}{0} & \textcolor{blue}{-1} & \textcolor{blue}{1} \\ 0 & \textcolor{blue}{2} & \textcolor{blue}{3} & \textcolor{blue}{1} \\ 0 & 0 & \textcolor{blue}{2} & \textcolor{blue}{-2} \\ 0 & 0 & 0 & \textcolor{blue}{1} \end{pmatrix}$$

Zuordnung farblich markiert:

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \textcolor{red}{1} & 1 & 0 & 0 \\ \textcolor{red}{1} & \textcolor{green}{0} & 1 & 0 \\ \textcolor{red}{1} & \textcolor{green}{0} & \textcolor{violet}{1.5} & 1 \end{pmatrix}$$

$$U = \begin{pmatrix} \textcolor{blue}{-1} & \textcolor{blue}{0} & \textcolor{blue}{-1} & \textcolor{blue}{1} \\ 0 & \textcolor{blue}{2} & \textcolor{blue}{3} & \textcolor{blue}{1} \\ 0 & 0 & \textcolor{blue}{2} & \textcolor{blue}{-2} \\ 0 & 0 & 0 & \textcolor{blue}{1} \end{pmatrix}$$

Interpretation LU für Gauß

Beispiel zeigt:

- U speichert das **Ergebnis** einer Gauß-Elimination als ZSF
- L speichert die multiplikativen **Faktoren** der Operationen einer Gauß-Elimination
 - Spaltenweise:
 - *erste Spalte*: Faktoren für Verwendung der *ersten* Zeile zur Eliminierung jedes Elements
 - *zweite Spalte*: Faktoren für Verwendung der *zweiten* Zeile zur Eliminierung jedes Elements.
 - *dritte Spalte*: Faktoren für Verwendung der *dritten* Zeile zur Eliminierung jedes Elements.
 - etc.

Frage

Für $A\vec{x} = \vec{b}$ mit $A = LU$

- Wenn U die ZSF nach Gauß darstellt, warum können wir dann nicht einfach direkt in U rückwärts einsetzen?
 - bzw. warum müssen wir erst mit L vorwärts einsetzen und dann mit U rückwärts?

Frage

Für $A\vec{x} = \vec{b}$ mit $A = LU$

- Wenn U die ZSF nach Gauß darstellt, warum können wir dann nicht einfach direkt in U rückwärts einsetzen?
 - bzw. warum müssen wir erst mit L vorwärts einsetzen und dann mit U rückwärts?
- Die umgeformte rechte Seite $\vec{b}^*$ fehlt. Wenn wir einfach die ursprüngliche rechte Seite $\vec{b}$ des LGS nehmen sind die Gauß-Operationen auf der rechten Seite nicht berücksichtigt!
- Vorwärts einsetzen in L liefert genau diese fehlende rechte Seite, d.h., wendet die Umformungsoperationen auf $\vec{b}$ an

$A = LL^T$ a.k.a. Cholesky Zerlegung

Cholesky Zerlegung

- Spezialfall der LU Zerlegung
- Wenn A
 - reellwertig, symmetrisch und
 - positiv definit ist (alle Eigenwerte >0)
- Dann
 - $A = LL^T$

Wenn LL^T anwendbar: meist **deutlich schneller** als z.B. LU !

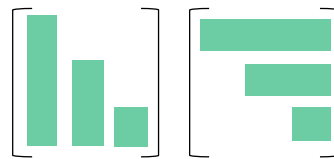


Bild basierend auf: Kenji Hiranabe. 2023. The Art of Linear Algebra.

Cholesky Zerlegung

- $A = LL^T$ ist meist effizienter als LU

Anmerkung 1: Für komplexe, hermetische A ergibt sich $A = LL^*$ (nur als Zusatzinfo, falls Ihnen diese Schreibweise begegnet, brauchen wir nicht)

Anmerkung 2: Positiv semidefinit ist auch möglich, in der Praxis numerisch manchmal schwieriger

Beispielrechnung

Mit $A\vec{x} = \vec{b}$,

$$A = LU = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 5 & 8 & 11 \\ 3 & 8 & 14 & 20 \\ 4 & 11 & 20 & 30 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 4 & 3 & 2 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix} \text{ und } \vec{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

Lösung für $\vec{y} = ?$, $\vec{x} = ?$

Ergebnis

$$\vec{y} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$L^T \vec{x} = \vec{y}$$

$$\vec{x} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Frage zum Beispiel

- Warum hätten wir die Lösung auch schon ohne Rechnung wissen können?

Frage zum Beispiel

- Warum hätten wir die Lösung auch schon ohne Rechnung wissen können?
- Rechte Seite $\vec{b}$ ist eine Spalte der Matrix A .
- Somit *muss* die Lösung der zugehörige Einheitsvektor sein.

Rückblick auf Lineare Regression

Lineare Regression

- Gesucht: Zusammenhang von $\vec{x} \in \mathbb{R}^n$ (z.B. Werbebudget) und y (z.B. erzielte Verkäufe)
- Lineares Modell

$$y = f(\vec{x}) = c_0 + c_1 \cdot x_1 + \dots + c_n x_n$$

- Umformulierung mit neuer Variable, die konstant $x_0 = 1$ ist

$$f(\vec{x}) = c_0 \cdot 1 + c_1 \cdot x_1 + \dots + c_n x_n = c_0 \cdot x_0 + c_1 \cdot x_1 + \dots + c_n x_n$$

Matrixschreibweise und Koeffizientenschätzung

- Lineares Modell: $\vec{y} = X\vec{c}$
 - Beobachtete Ausgabe $\vec{y}$ im Trainingsdatensatz
 - 'Design-Matrix' mit X -Werten (inkl. konstanter x_0)
- Bestimmung der Koeffizienten:

$$\vec{c} = (X^T X)^{-1} X^T \vec{y}$$

Wie lösen wir das?

$$\vec{c} = (X^T X)^{-1} X^T \vec{y}$$

- Hilft uns vielleicht $LU, LL^T, CR, \dots$?
- Welche Eigenschaften hat $(X^T X)$?

Wie lösen wir das?

$$\vec{c} = (X^T X)^{-1} X^T \vec{y}$$

- Hilft uns vielleicht $LU, LL^T, QR, CR, \dots$?
- Welche Eigenschaften hat $(X^T X)$?
 - Symmetrisch!
 - Zumindest positiv semi-definit, im Idealfall positiv definit.

→ LL^T Zerlegung (Cholesky)!

(Aber auch andere sind möglich, insbesondere QR oder SVD)

R-Lab

Beispieldaten

```

1 b <- 1:4
2 G <- matrix(c(1,4,5,1,0,1,-2,1,0,0,1,3,0,0,0,1),4,4)
3 A <- G %*% t(G)
4 A

```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	5	1
[2,]	4	17	18	5
[3,]	5	18	30	6
[4,]	1	5	6	12

Lösen mit solve (blackbox)

Inverse:

```
1 solve(A)
```

```
      [,1] [,2] [,3] [,4]  
[1,] 1950 -324 -139  42  
[2,] -324   54  23  -7  
[3,] -139  23  10  -3  
[4,]   42  -7  -3   1
```

Lösung für $\vec{x}$:

```
1 solve(A,b)
```

```
[1] 1053 -175 -75  23
```


Lösen mit $(P)LU$

```
1 res <- Matrix::lu(A)
2 rese <- Matrix::expand(res)
3 (L <- rese$L)
```

```
4 x 4 Matrix of class "dtrMatrix" (unitriangular)
      [,1]      [,2]      [,3]      [,4]
[1,] 1.00000000 .                .                .
[2,] 0.80000000 1.00000000 .                .
[3,] 0.20000000 0.53846154 1.00000000 .
[4,] 0.20000000 0.15384615 -0.02380952 1.00000000
```

```
1 (U <- rese$U)
```

```
4 x 4 Matrix of class "dtrMatrix"
      [,1]      [,2]      [,3]      [,4]
[1,] 5.00000000 18.00000000 30.00000000 6.00000000
[2,] .          2.60000000 -6.00000000 0.20000000
[3,] .          .          3.23076923 10.69230769
[4,] .          .          .          0.02380952
```

```
1 (P <- rese$P) #Permutation
```

```
4 x 4 sparse Matrix of class "pMatrix"
```

```
[1,] . . . |
[2,] . | . .
[3,] | . . .
[4,] . . | .
```

Lösen mit $(P)LU$

```
1 A - P %*% L %*% U #Prüfen
```

```
4 x 4 Matrix of class "dgeMatrix"
```

```
  [,1] [,2] [,3] [,4]
[1,]   0   0   0   0
[2,]   0   0   0   0
[3,]   0   0   0   0
[4,]   0   0   0   0
```

```
1 iterative_sorting <- function(vector, perm) {
2   for (i in 1:length(perm))
3     vector[c(i, perm[i])] <- vector[c(perm[i], i)]
4   return(vector)
5 }
6 LUr <- Matrix::lu(A)
7 LUres <- Matrix::expand(LUr)
8 bp <- iterative_sorting(b, LUr@perm)
9 y <- forwardsolve(LUres$L, bp)
10 (x <- backsolve(LUres$U, y))
```

```
[1] 1053 -175 -75 23
```

Lösen mit Cholesky

Achtung: `chol` liefert hier obere, nicht untere Dreiecksmatrix.

```
1 L <- chol(A)
2 y <- forwardsolve(t(L), b)
3 y
```

```
[1] 1 -2 -6 23
```

```
1 x <- backsolve(L, y)
2 x
```

```
[1] 1053 -175 -75 23
```

```
1 A - t(L) %*% L #Prüfen. Achtung Reihenfolge!
```

	[,1]	[,2]	[,3]	[,4]
[1,]	0	0	0	0
[2,]	0	0	0	0
[3,]	0	0	0	0
[4,]	0	0	0	0

Lösen mit QR

```
1 x <- qr.solve(A,b)
2 x
```

```
[1] 1053 -175 -75 23
```

Benchmark (ohne LGS Lösung)

```
1 require(microbenchmark)
2 microbenchmark(solve(A))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
solve(A)		9.801	10.002	10.79618	10.102	10.302	49.902	100

```
1 microbenchmark(Matrix::lu(A))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
Matrix::lu(A)		7.5	7.8015	8.89896	8.501	8.9005	51.1	100

```
1 microbenchmark(chol(A))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
chol(A)		5.401	5.5	5.81502	5.501	5.601	27.401	100

```
1 microbenchmark(qr(A))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
qr(A)		9.701	10.1	10.80797	10.201	10.501	52.301	100

Benchmark solve (mit LGS Lösung)

```
1 ex <- expression({  
2   x <- solve(A,b)  
3 })  
4 microbenchmark(eval(ex))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
eval(ex)		8.401	8.601	9.56997	8.8005	9.1015	42.1	100

Benchmark LU (mit LGS Lösung)

```

1  iterative_sorting <- function(vector, perm) {
2      for (i in 1:length(perm)) {
3          vector[c(i, perm[i])] <- vector[c(perm[i], i)]
4      }
5      return(vector)
6  }
7  ex <- expression({
8      LUr <- Matrix::lu(A)
9      LUres <- Matrix::expand(LUr)
10     bp <- iterative_sorting(b, LUr@perm)
11     y <- forwardsolve(LUres$L, bp)
12     x <- backsolve(LUres$U, y)
13 })
14 microbenchmark(eval(ex))

```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
eval(ex)		313.501	324.401	360.093	328.6515	336.5015	3008.201	100

Benchmark Chol (mit LGS Lösung)

```
1 ex <- expression({  
2   L <- chol(A)  
3   y <- forwardsolve(t(L), b)  
4   x <- backsolve(L, y)  
5 })  
6 microbenchmark(eval(ex))
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
eval(ex)		27.801	28.702	30.87009	29.051	29.8005	88.301	100

Benchmark qr (mit LGS Lösung)

```
1 ex <- expression({  
2   x <- qr.solve(A, y)  
3 })  
4 microbenchmark(eval(ex))
```

Unit: microseconds

expr	min	lq	mean	median	uq	max	neval
eval(ex)	30.6	31.201	32.65097	31.701	32.301	93.301	100

Zwischenfazit

- So gut wie nie Inverse direkt berechnen
- Am besten immer mit realistischen Tests prüfen (Benchmarks!), welche Variante am besten Funktioniert
- Dabei auch Einfluss der Matrixgröße berücksichtigen

Advertising

```
1 ads <- read.csv("../GrundLinAlg/data/Advertising.csv", row.names=1)
2 A <- as.matrix(ads)
3 head(A)
```

	TV	radio	newspaper	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9
6	8.7	48.9	75.0	7.2

Vorbereitung für Lineare Regression

```
1 X <- cbind(1,A[,1:3])
2 y <- A[,4]
3 XT <- t(X)
4 XTX <- XT %*% X
5 XTy <- XT %*% y
```

Lineares Modell: $\vec{y} = X\vec{c}$

mit: $\vec{c} = (X^T X)^{-1} X^T \vec{y}$

Prüfen ob XTX symmetrisch ist

Wir wissen ja bereits, dass XTX reellwertig sein muss.

```
1 all.equal(XTX,t(XTX))
```

```
[1] TRUE
```

```
1 isSymmetric(XTX)
```

```
[1] TRUE
```

Prüfen ob XTX positiv definit ist

```
1 eigs <- eigen(XTX)$values
2 eigs
```

```
[1] 6.027836e+06 1.588273e+05 3.783007e+04 2.919663e+01
```

```
1 eigs > 0
```

```
[1] TRUE TRUE TRUE TRUE
```

Also: XTX ist reell, symmetrisch und PD

Koeffizienten mit Cholesky

$$\vec{c} = (X^T X)^{-1} X^T \vec{y}$$

```
1 L <- chol(XTX)
2 y <- forwardsolve(t(L), XTy)
3 (c <- backsolve(L, y))
```

```
      [,1]
[1,] 2.938889369
[2,] 0.045764645
[3,] 0.188530017
[4,] -0.001037493
```

Vergleich:

```
1 lm(sales~.,data=ads)$coef
```

```
(Intercept)      TV      radio  newspaper
2.938889369  0.045764645  0.188530017 -0.001037493
```